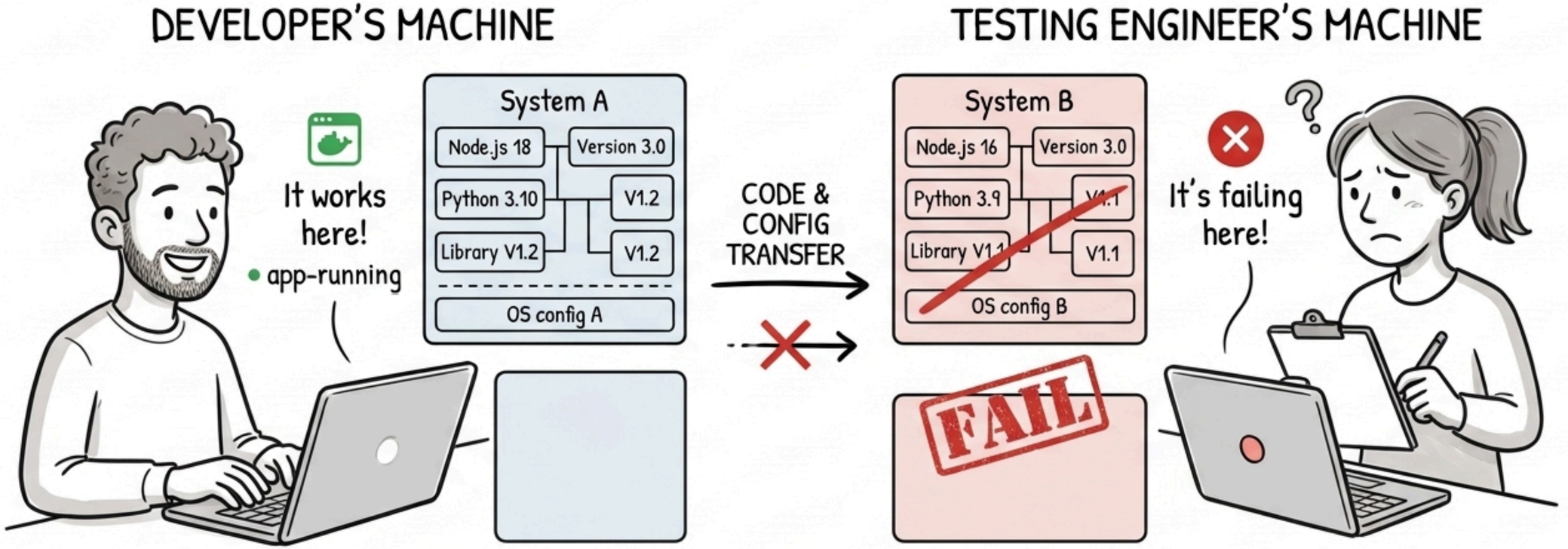
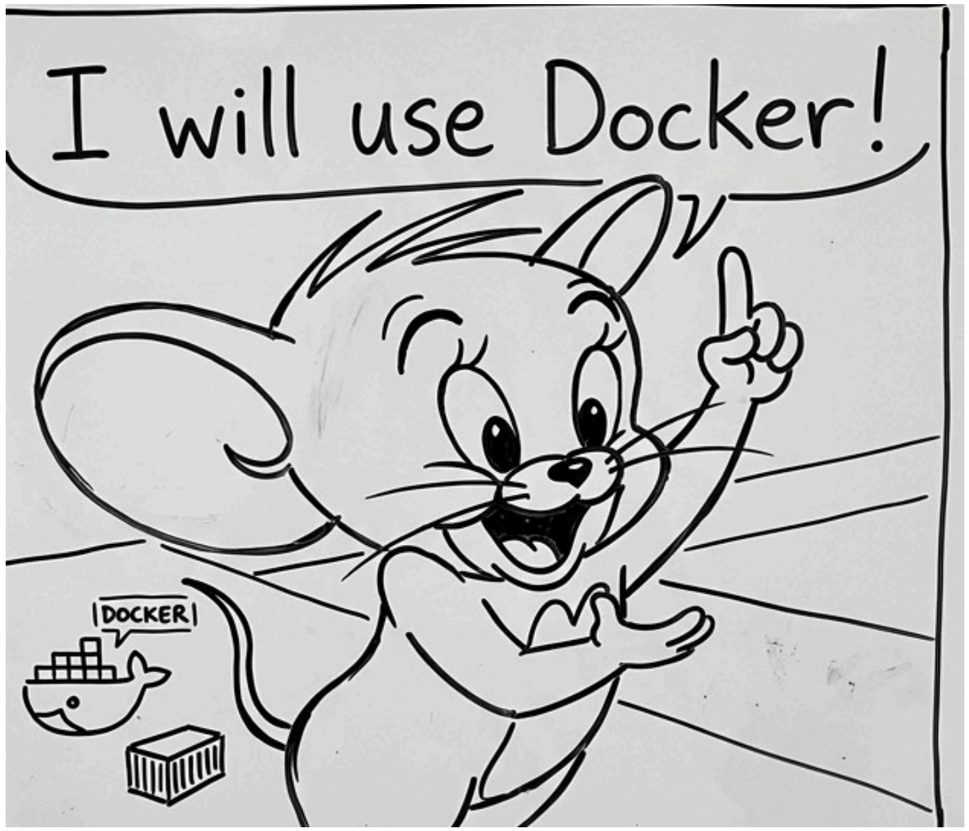
THE 'WORKS ON MY MACHINE' PROBLEM



THE DOCKER SOLUTION:
Standardized Environments


I will use Docker!
DOCKER

WHAT IS DOCKER?

(1) Input

Application Code

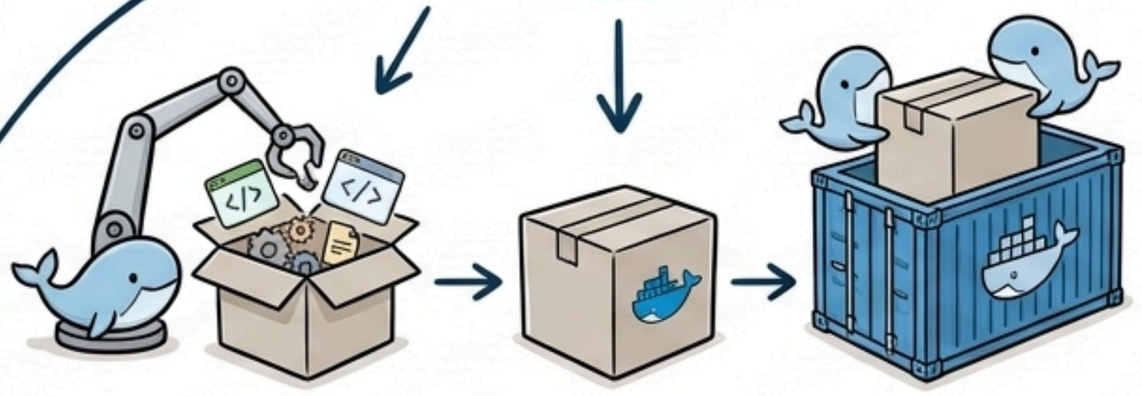
Libraries

System Dependencies

Config

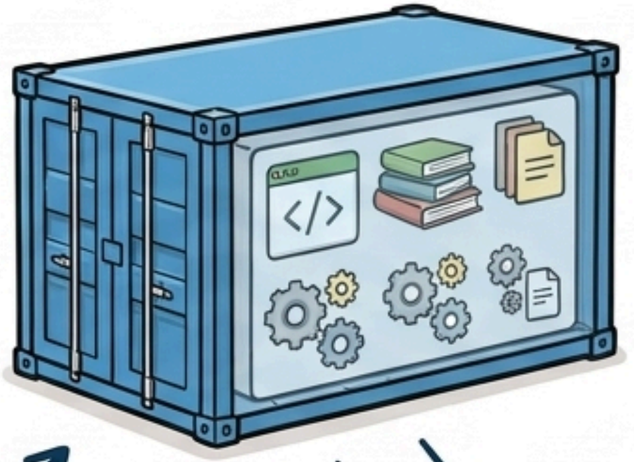
(2) Engine & Containerization

DOCKER ENGINE



(3) Output

PACKAGED CONTAINER



ANY CLOUD ANY SERVER ANY OS

Runs Consistently Anywhere

Typing area

THE DOCKER CONCEPT:
APPLICATION + LIBRARIES + DEPENDENCIES = A STANDARD CONTAINER

WHAT IS DOCKER?

- Docker is an open-source containerization platform that packages an application along with its dependencies, libraries, and configuration files into a container.

- A Docker container provides the same runtime environment everywhere, ensuring the application runs consistently across Development, Testing, and Production environments.

DOCKER INSTALLATION

Docker Installation on EC2:

Connect to EC2 using SSH (Port 22)
Install Docker using yum install docker -y
Start Docker service
Enable Docker on system reboot
Verify using docker --version

Ports

22 → SSH Access
80 → HTTP Applications
443 → HTTPS Applications

Install Docker

sudo yum install docker -y

Start Docker Service

sudo systemctl start docker

Enable Docker at Boot

sudo systemctl enable docker

Verify Installation

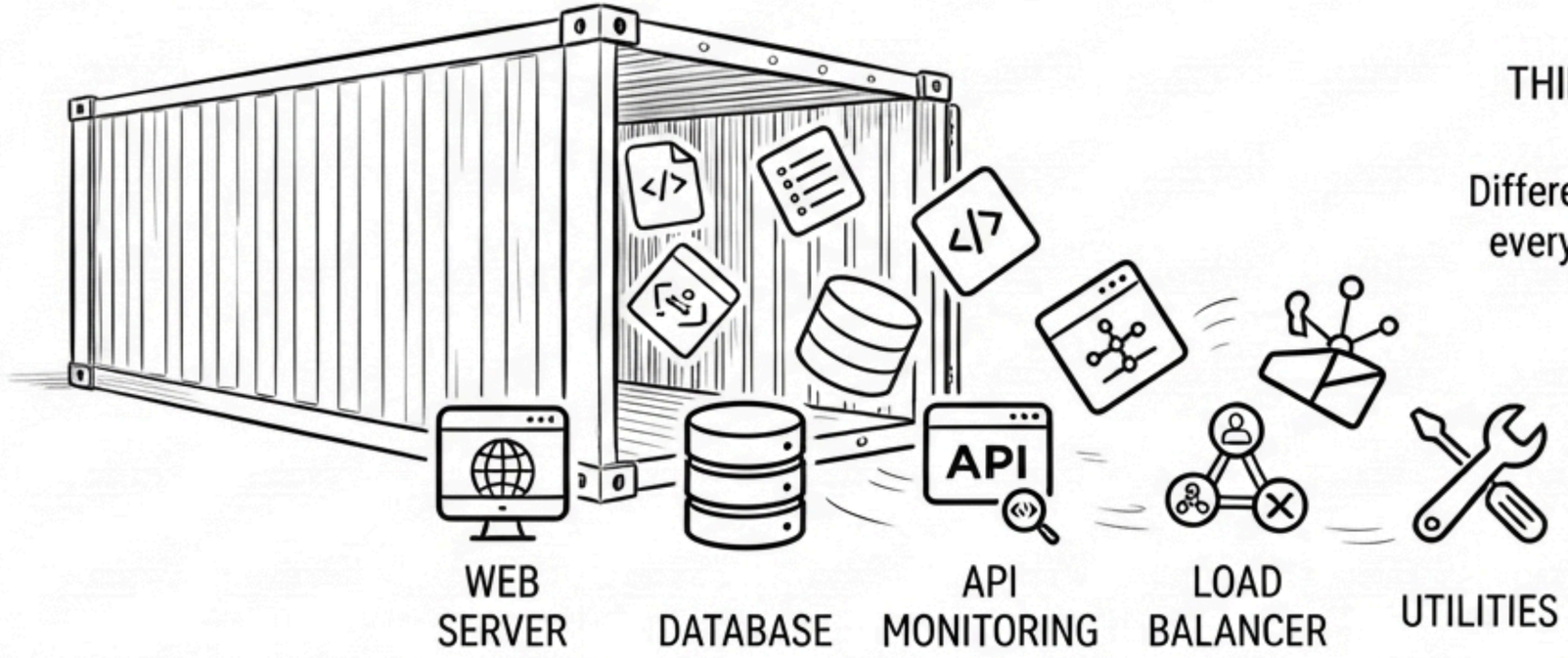
docker --version
docker ps

Test Docker

docker run hello-world

WHAT IS A CONTAINER?

A container is a lightweighted isolated package that contains an application along with its libraries, dependencies, and configurations required to run it.



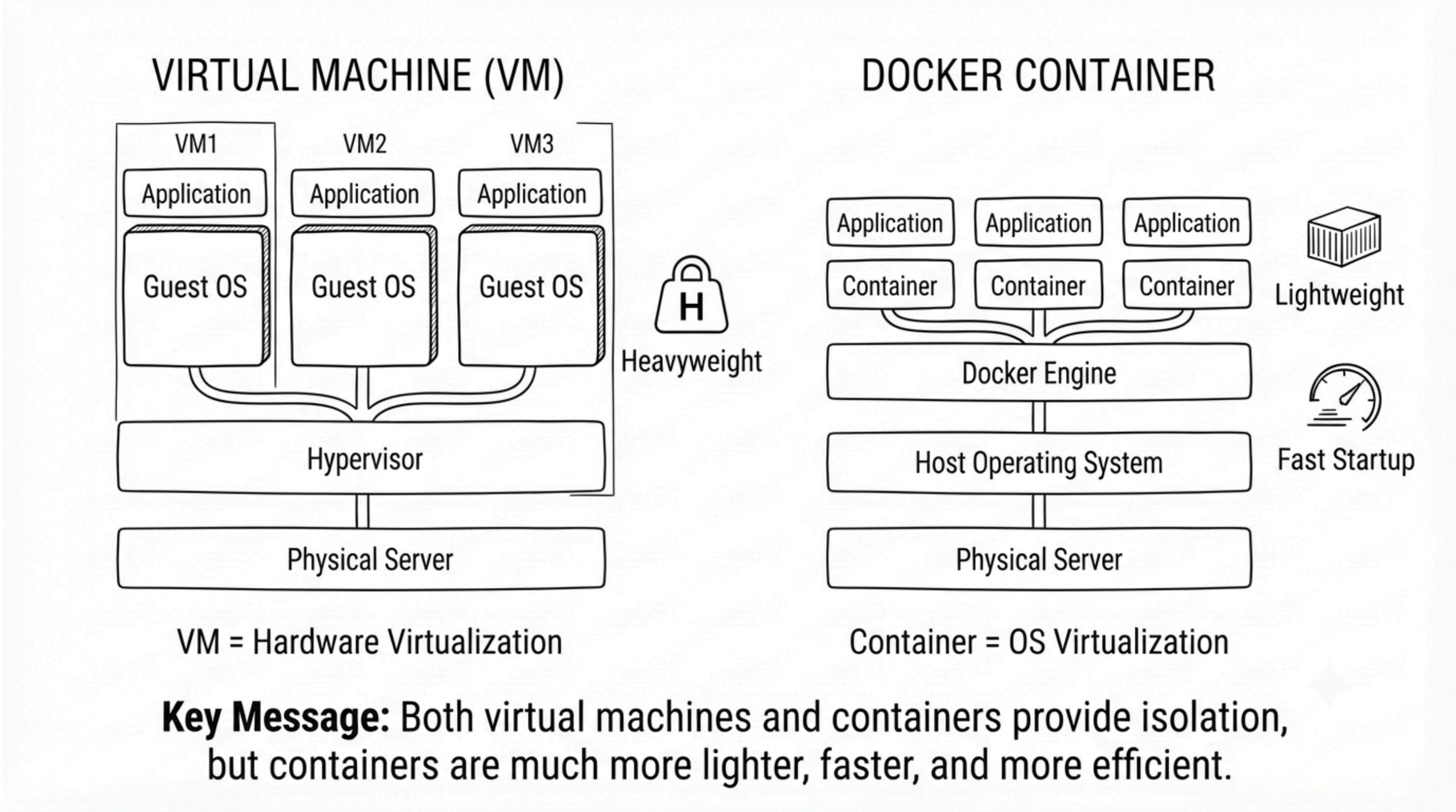
THE SHIPPING CONTAINER ANALOGY

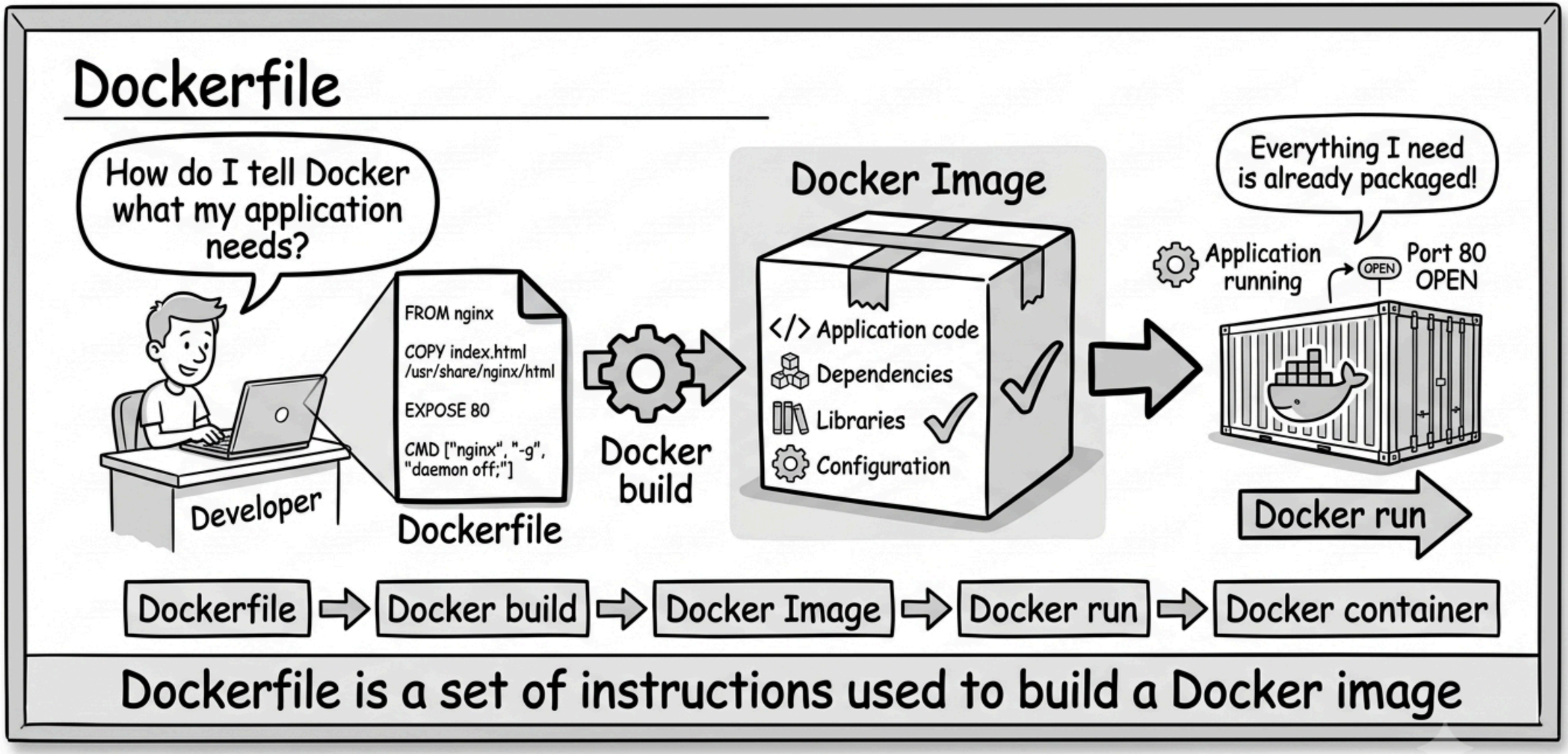


THINK OF A CONTAINER LIKE A SHIPPING CONTAINER:
Different goods can be packed, protects everything, moved anywhere, content remains unchanged.

PORTABLE • LIGHTWEIGHT • CONSISTENT • ISOLATED • FAST TO DEPLOY

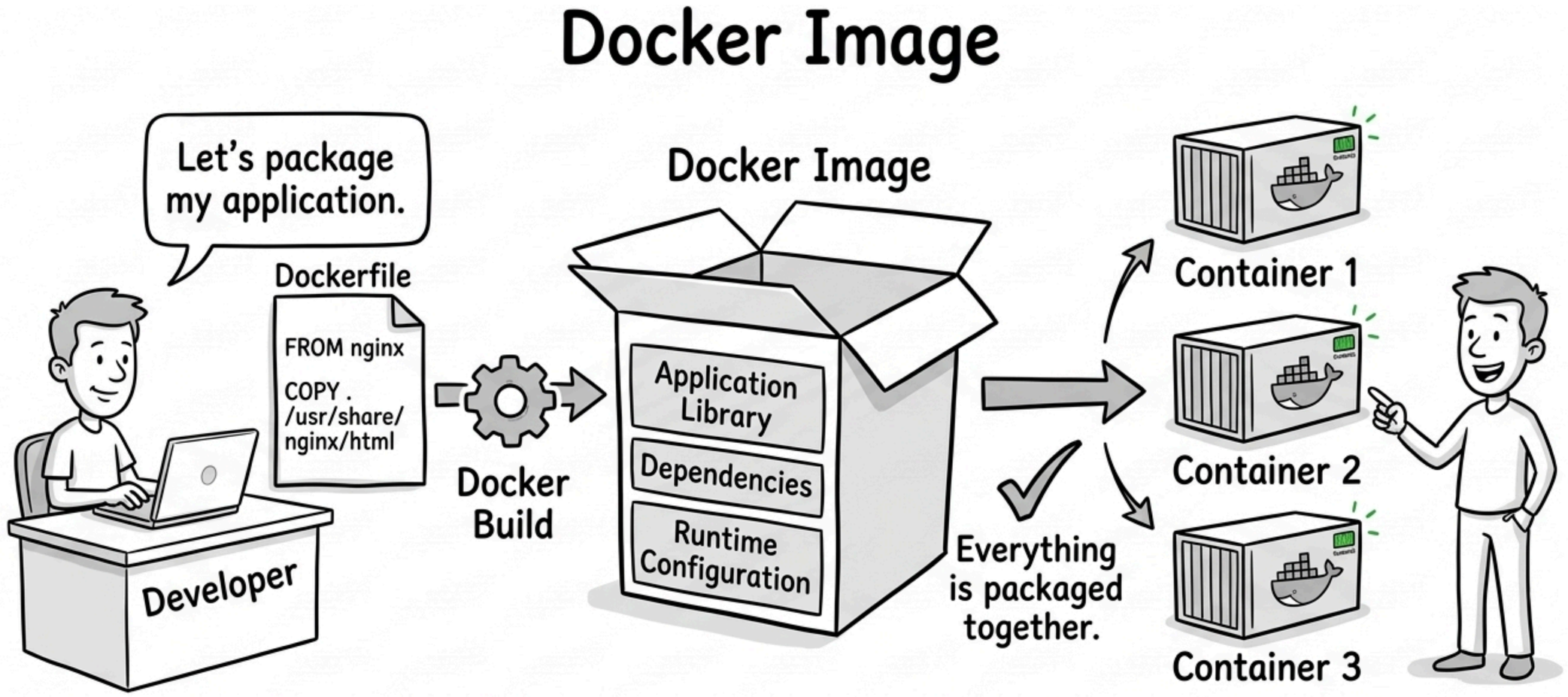
Virtual Machines virtualize hardware and require a Guest OS, while Containers virtualize the operating system and share the Host OS, making them lighter and faster.





What is the Dockerfile?
A Dockerfile is a text file that contains a set of instructions for building a Docker image. It defines all the necessary steps and dependencies required to create a layered, runnable Docker image.

Dockerfile → Docker Image → Docker Container



Dockerfile ⇨ Docker Build ⇨ Docker Image ⇨ Docker Run ⇨ Containers

One Docker image can create multiple containers.

Docker Workflow

Create a Dockerfile

Build an Image using docker build

Run a Container using docker run

Manage container lifecycle (start, stop, restart, remove)

STEPS:

Create Dockerfile

vim Dockerfile

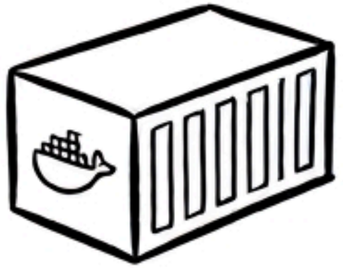
Build Image

docker build -t myapp:v1 .

Run Container

docker run -d -p 80:80 --name mycontainer myapp:v1

DOCKER CONTAINER LIFECYCLE



CREATED

CONTAINER IS CREATED

APPLICATION IS ACTIVE & EXECUTING

STARTED

APPLICATION IS ACTIVE

RUNNING APPLICATION IS ACTIVE

RUNNING

RESUMES EXECUTION

EXECUTION TEMPORARILY SUSPENDED

PAUSED

CONTAINER IS DELETED

STOPPED

CONTAINER STARTS AGAIN

CONTAINER IS RESTARTED FROM SAVED STATE

RESTARTED

RUNNING APPLICATION IS ACTIVE

CONTAINER STARTS AGAIN

RUNNING

CONTAINER IS DELETED

REMOVED

CONTAINER IS NOT RUNNING

Dockerfile:

A Dockerfile is a text file that contains instructions to build a Docker Image automatically.

✅ Automates image creation

✅ Defines the application environment

✅ Used with docker build

Docker Image:

A Docker Image is a read-only template that contains an application and everything required to run it.

✅ Contains application and dependencies

✅ Used to create containers

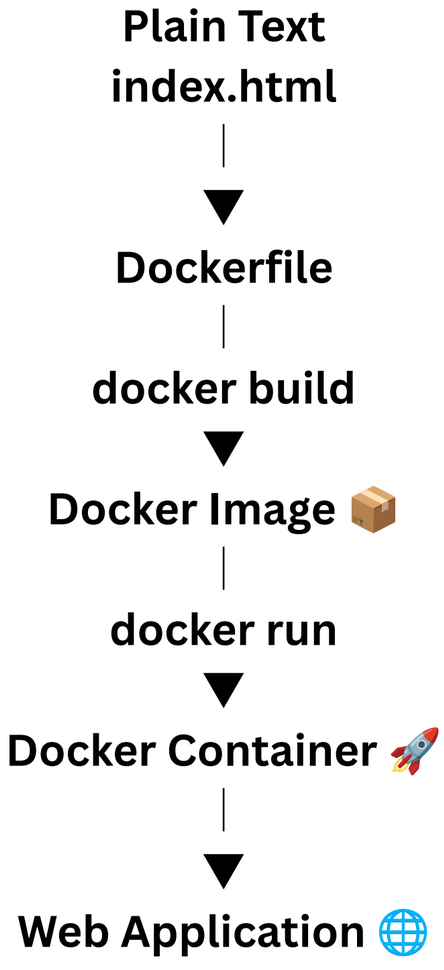
✅ Portable and reusable

Building an Nginx Application Using Docker

Steps:

1. Create the application file (index.html)
2. Define build instructions in a Dockerfile
3. Build a Docker Image using docker build
4. Run the image as a Docker Container
5. Access the application through a web browser

WORKFLOW:



Dockerfile:

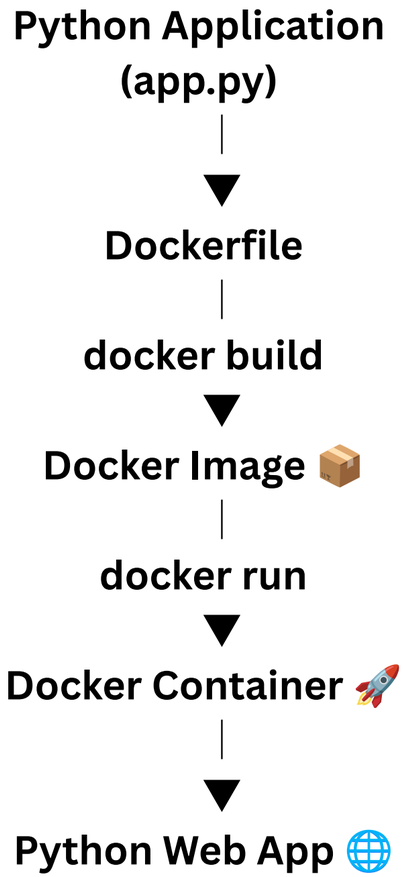
```
FROM nginx:latest
COPY index.html   /usr/share/nginx/html/
EXPOSE 80
```

Building a Python Application Using Docker

Steps

1.Create the Python application file (app.py)
2.Define build instructions in a Dockerfile
3.Build a Docker Image using docker build
4.Run the image as a Docker Container
5.Access the application through a web browser

WORKFLOW:



DOCKER FILE:

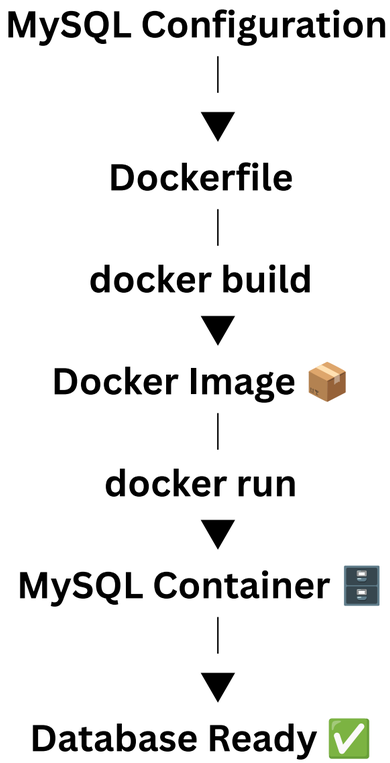
```
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

Deploying a MySQL Database Using Docker

Steps

1. Use the official MySQL base image
2. Configure database settings using environment variables
3. Build (optional) or pull the MySQL image
4. Run the image as a Docker Container
5. Connect applications to the database

WORKFLOW:



DOCKER FILE:

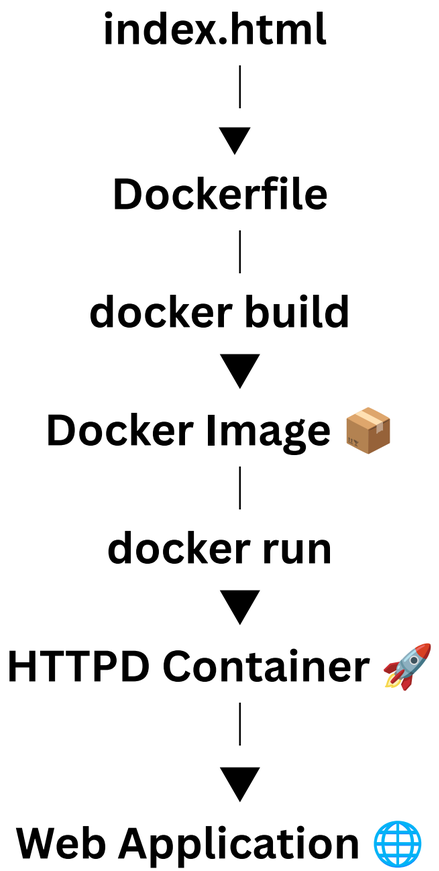
```
FROM mysql:8.0
ENV MYSQL_ROOT_PASSWORD=root123
ENV MYSQL_DATABASE=mydb
EXPOSE 3306
`
```

Deploying an Apache (HTTPD) Application Using Docker

Steps

1. Create the application file (index.html)
2. Define build instructions in a Dockerfile
3. Build a Docker Image using docker build
4. Run the image as a Docker Container
5. Access the application through a web browser

WORKFLOW:



DOCKER FILE:

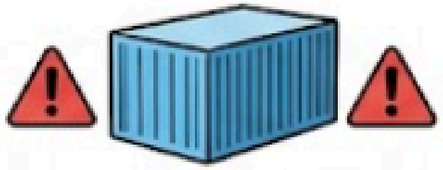
```
FROM httpd:latest
COPY index.html /usr/local/apache2/htdocs/
EXPOSE 80
```

WHY PRODUCTION-GRADE DOCKERFILES?

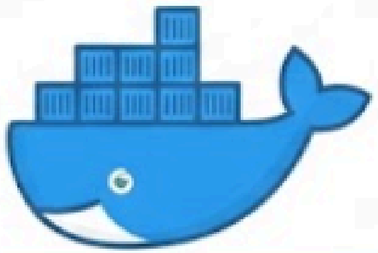


'Deploy to Production'





- ✗ Running as Root User
- ✗ Large Image Size
- ✗ Unnecessary Packages
- ✗ Security Risks



PRODUCTION-GRADE DOCKERFILE

Secure • Lightweight • Reliable • Production Ready



- ✓ Non-Root User
- ✓ Lightweight Image
- ✓ Secure Configuration
- ✓ Optimized Build
- ✓ Faster Deployment

P**roduction-Grade Dockerfile**

A production-grade Dockerfile is designed for security, performance, and reliability
in real-world environments.

Best Practices:

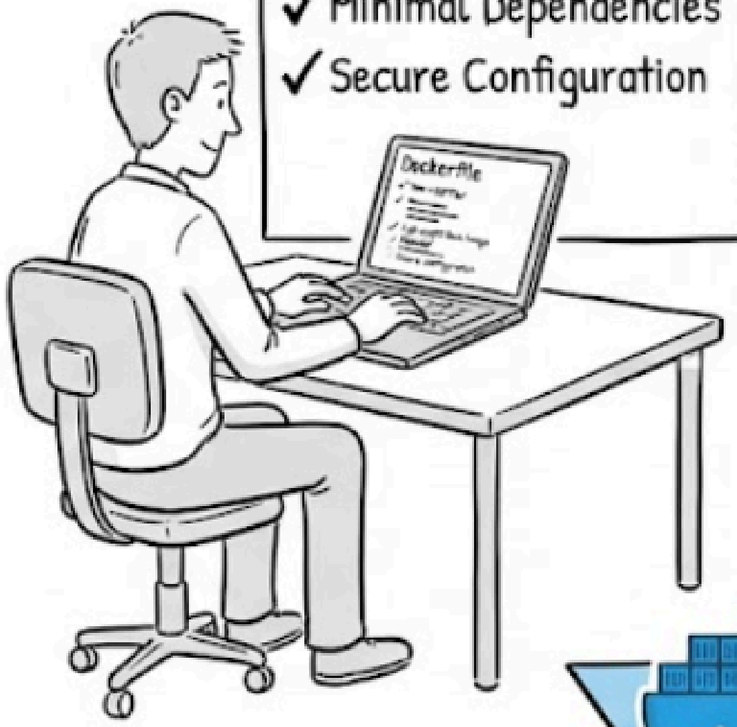
Use lightweight base images (Alpine/Slim)
Run containers as non-root users
Install only required dependencies
Remove unnecessary files and packages
Follow the principle of least privilege
Optimize image size for faster deployments

✅ More Secure
✅ Smaller Images
✅ Faster Deployments
✅ Easier Maintenance

PRODUCTION-GRADE DOCKERFILE

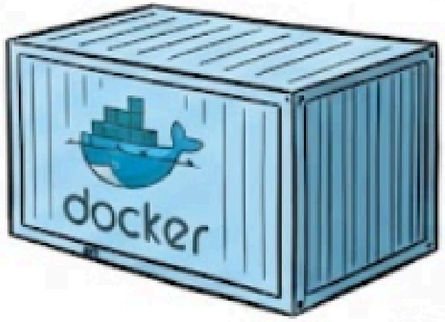
Developer

- ✓ Non-root User
- ✓ Lightweight Base Image
- ✓ Minimal Dependencies
- ✓ Secure Configuration



✓ Security Check

✓ Least Privilege Check



✓ Size Optimization

✓ Production Standards

Production Engineer

✓ Deployment Success



- ✓ Deployment Success
- ✓ Secure Container
- ✓ Stable Application

PRODUCTION READY

Secure • Lightweight • Reliable

Components of a Production-Grade Dockerfile

📦 **Base Image**: Defines the foundation of the container by providing the operating system and runtime environment required by the application.

📁 **Working Directory**: Specifies the default location inside the container where commands and application processes are executed.

📋 **Application Files:** Includes the source code, configuration files, and other resources required for the application to run.

🔧 **Dependency Installation:** Installs the libraries, packages, and runtime dependencies needed by the application.

🌐 **Port Configuration:** Documents the network ports that the application uses to communicate with external systems.

👤 **User Configuration:** Defines the user account under which the application runs, improving security and access control.

🚀 **Startup Command:** Specifies the command or process that is executed when the container starts.

🏗️ **Build Configuration:** Manages application compilation, packaging, and artifact generation during the image build process.

🔒 **Security Configuration:** Implements security best practices such as least-privilege access, reduced attack surface, and secure settings.

⚡ **Multi-Stage Build**: Separates build and runtime environments to create smaller, cleaner, and more efficient container images.

NGINX Production Dockerfile

Dockerfile

```
FROM nginx:alpine
RUN rm -rf /usr/share/nginx/html/*
COPY . /usr/share/nginx/html/
RUN chown -R nginx:nginx /usr/share/nginx/html
EXPOSE 80
USER nginx
CMD ["nginx", "-g", "daemon off;"]
```

Key Points

Alpine image for smaller size

Removes default content

Uses non-root nginx user

Secure and lightweight

Apache Production Dockerfile

Dockerfile

```
FROM httpd:2.4-alpine
RUN rm -rf /usr/local/apache2/htdocs/*
COPY --chown=daemon:daemon . /usr/local/apache2/htdocs/
EXPOSE 80
USER daemon
CMD ["httpd-foreground"]
```

Key Points

Lightweight Alpine base

Correct file ownership

Runs as daemon user

Production-ready web server

Python Production Dockerfile

DOCKER FILE:

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
PYTHONUNBUFFERED=1 \
PIP_NO_CACHE_DIR=1
WORKDIR /app
COPY req.txt .
RUN pip install --no-cache-dir -r req.txt
COPY --chown=10001:10001 . .
RUN useradd \
--uid 10001 \
--create-home \
appuser
USER 10001
EXPOSE 8000
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "app:app"]
```

KEY POINTS:

Key Points

- Slim base image
- No pip cache
- Dedicated application user
- Uses Gunicorn for production

NODE js production grade Docker File:

PRODUCTION FILE:

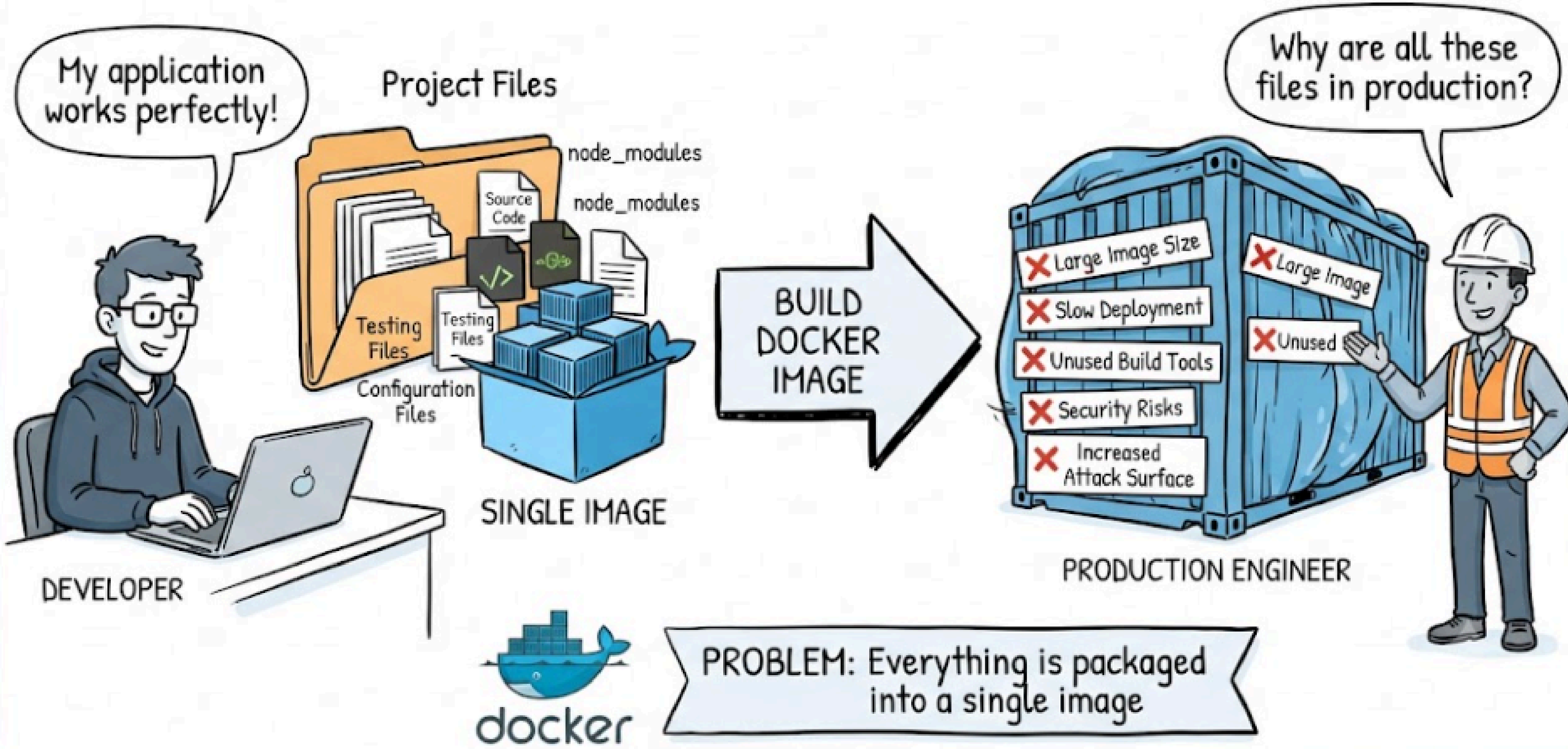
```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev && \
npm cache clean --force
COPY --chown=node:node . .
USER node
EXPOSE 3000
CMD ["node", "index.js"]
```

Key Points

- Installs only production dependencies
- Clears npm cache
- Uses non-root node user
- Optimized image size

}

WHY MULTI-STAGE DOCKERFILES?



Multi-Stage Dockerfile

A Multi-Stage Dockerfile uses multiple stages within a single Dockerfile to separate the build process from the runtime environment.

Instead of including build tools, source code, and development dependencies in the final image, only the required application artifacts are copied to the production image.

Benefits

✅ Smaller Image Size

✅ Better Security

✅ Faster Deployments

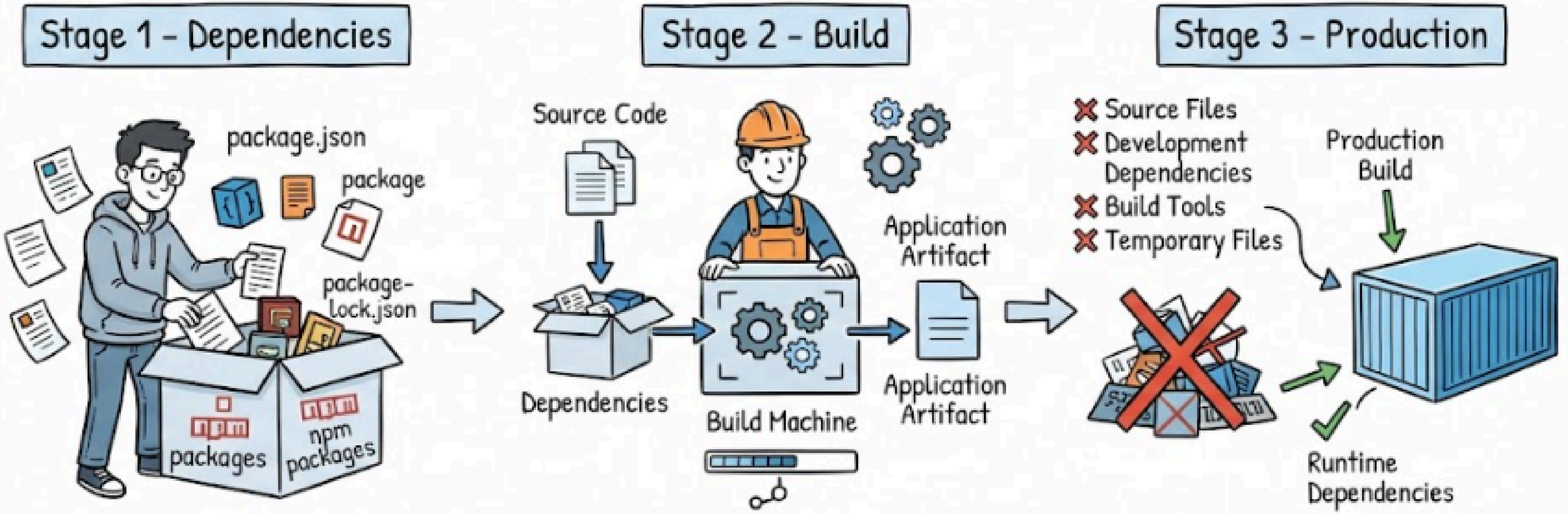
✅ Reduced Attack Surface

✅ Cleaner Production Images

Flow

Dependencies Stage → Build Stage → Production Stage

MULTI-STAGE DOCKERFILE



docker

MULTI-STAGE BUILD

- ✓ Smaller Image Size
- ✓ Faster Deployment
- ✓ Faster Deployment
- ✓ Better Security
- ✓ Reduced Attack Surface
- ✓ Production Ready

Multi-Stage Dockerfile Example

A Multi-Stage Dockerfile divides the build process into separate stages, ensuring that only the required files are included in the final production image.

Step 1: Dependencies Stage

Dockerfile

```
FROM node:22-alpine as dependencies
WORKDIR /app
COPY package*.json ./
RUN npm ci
```

Creates a dedicated dependencies stage.
Installs all required npm packages.
Stores dependencies for later stages.

Step 2: Build Stage

Dockerfile

```
FROM node:22-alpine as build
WORKDIR /app
COPY --from=dependencies /app/node_modules ./node_modules
COPY . .
RUN npm run build
```

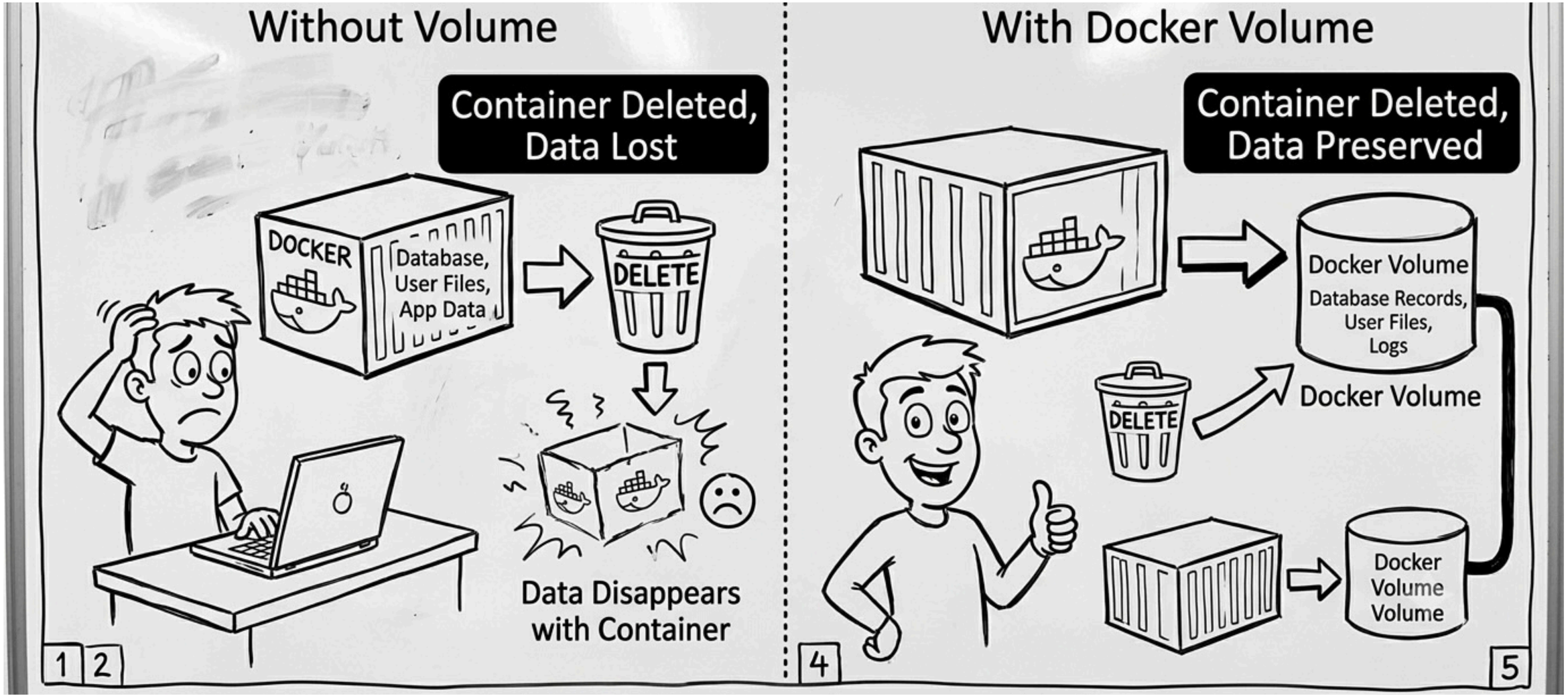
Copies source code and dependencies.
Builds the application.
Generates production-ready artifacts.

Step 3: Production Stage

Dockerfile

```
FROM node:22-alpine as prod
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
EXPOSE 3000
USER node
CMD ["node", "dist/server.js"]
```

DOCKER VOLUMES



Docker Volumes

Definition

Docker Volumes are a mechanism for storing and managing persistent data outside the container's filesystem. Data stored in a volume remains available even if the container is stopped, removed, or recreated.

Why Use Docker Volumes?

✅ Prevent data loss when containers are deleted

✅ Provide persistent storage for databases and application data

✅ Enable data sharing between multiple containers

✅ Improve data management and portability

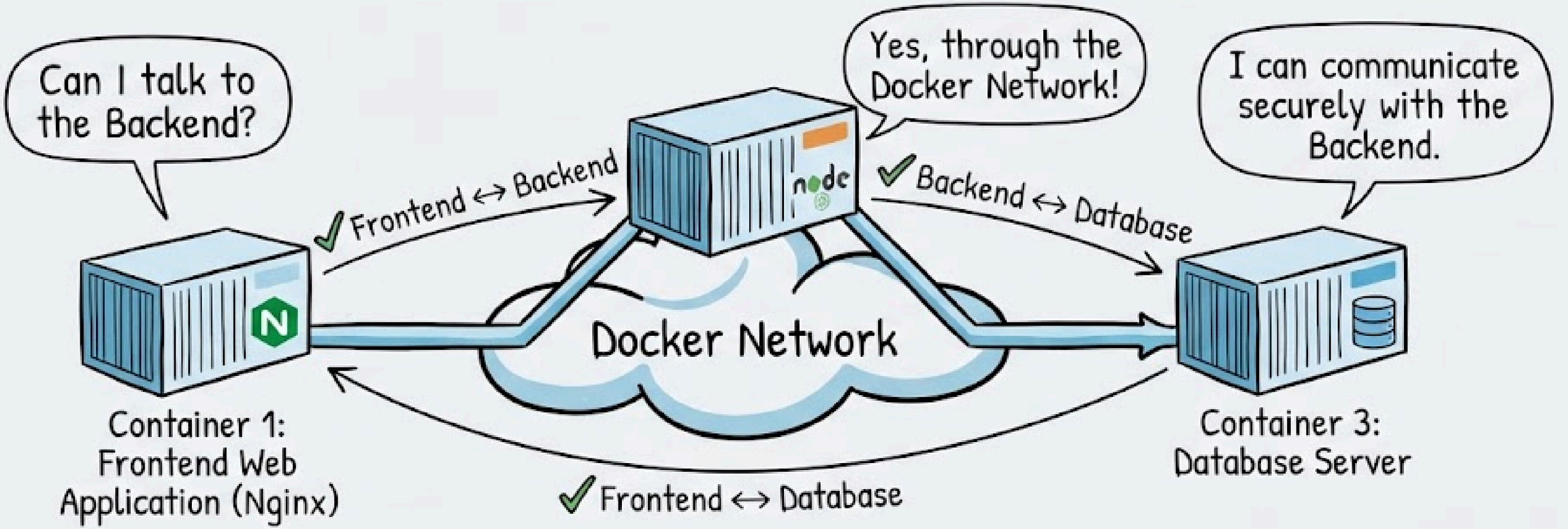
Key Takeaway

Containers are temporary, but Docker Volumes ensure that important data remains permanent and accessible.

Docker Volume Commands

Create a volume
docker volume create myvolume
List volumes
docker volume ls
Inspect a volume
docker volume inspect myvolume
Attach volume to a container
docker run -d \ -v myvolume:/app/data \ --name mycontainer nginx
Remove a volume
docker volume rm myvolume
Remove unused volumes
docker volume prune

DOCKER NETWORKING



✓ Container-to-Container Communication ✓ Network Isolation

✓ Service Discovery ✓ Secure Connectivity

Frontend ↔ Backend ↔ Database

Docker Network

Docker Networking

Docker Networking enables containers to communicate with each other through isolated virtual networks.

Key Benefits

Container-to-container communication

Service discovery using container names

Network isolation and security

Simplified multi-container deployments

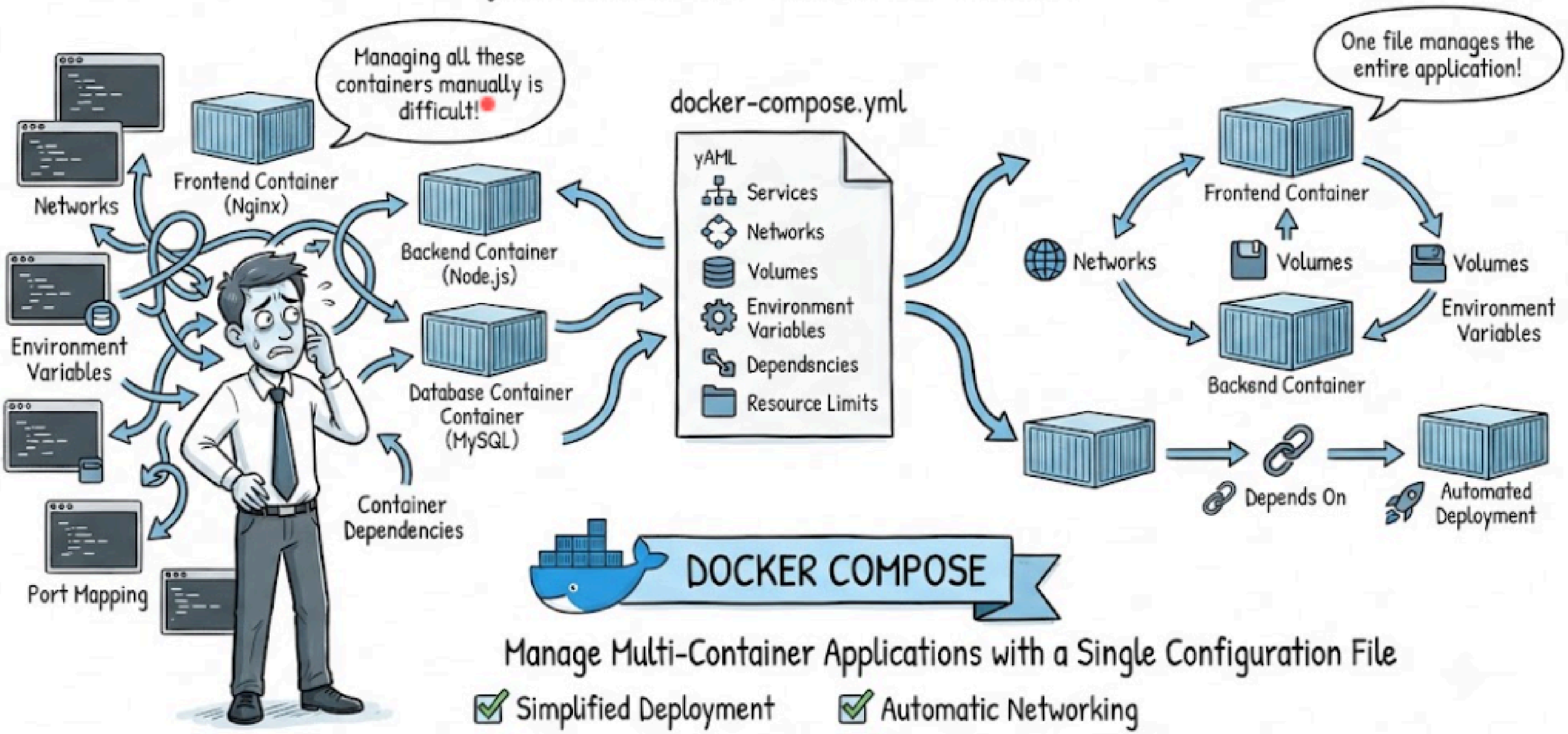
Controlled external access

Example: Frontend Container → Backend Container → Database Container

Key Takeaway: Docker Networking acts as a virtual bridge that securely connects containers and allows seamless communication between services.

🚀 Result: Secure, reliable, and scalable container communication

DOCKER COMPOSE



DOCKER COMPOSE

Manage Multi-Container Applications with a Single Configuration File

- ✅ Simplified Deployment
- ✅ Automatic Networking
- ✅ Centralized Configuration
- ✅ Faster Environment Setup

Why Docker Compose?

Managing a multi-container application manually can become complex and time-consuming.

Challenges

❌ Multiple docker run commands
❌ Manual network configuration
❌ Managing volumes separately
❌ Handling environment variables individually
❌ Tracking service dependencies
❌ Difficult setup and maintenance

Example

Frontend Container
↓
Backend Container
↓
Database Container

Each container requires separate configuration and management.

Solution

Docker Compose simplifies the process by managing the entire application stack using a single docker-compose.yml file.

🚀 One File • One Command • Complete Application Management

This is the perfect amount of content for a PPT slide before the actual "Docker Compose" introduction slide.

Docker Compose Example

A Docker Compose file defines and manages multiple services, networks, volumes, and application configurations from a single YAML file.

Services Defined : Services represent the individual containers that make up the application.

Frontend Service:Handles user interaction and client requests.

Container: cont1

Image: shaikmustafa/dm

Port: 1111:80

Uses Volume & Network

Backend Service : Processes business logic and communicates with the database

Container: cont2

Image: shaikmustafa/cycle

Port: 1112:80

Uses Environment Variables

Depends on Database

Database Service: Stores and manages application data.

Container: cont3

Image: shaikmustafa/mygame

Port: 1113:80

Uses Dedicated Volume

Architecture

Frontend → Backend → Database

🔗 Dependencies

Docker Compose uses the depends_on directive to define relationships between services.

- Ensures services start in the required sequence.
- In this example, the Backend service depends on the Database service.
- Helps maintain proper application startup order.

🚀 Resource Management

Resource limits help control CPU and memory usage for each container.

- Reservations guarantee minimum resources required by a service.
- Limits prevent containers from consuming excessive system resources.
- Improves application stability and efficient resource utilization.

Result

✅ Centralized Configuration
✅ Automated Networking
✅ Persistent Storage
✅ Service Dependency Management
✅ Simplified Multi-Container Deployment

Complete Docker Compose Example

Services

frontend (cont1)

backend (cont2)

database (cont3)

Networking

network1 (Bridge)

network2 (Bridge)

network3 (Bridge)

Storage

volume1 → frontend data

volume2 → backend data

volume3 → database data

Environment Configuration

External Environment File (dev.env)

Environment Variables

APP_ENV

APP_PORT

DB_HOST

DB_PORT

Resource Management

CPU Reservation: 0.5

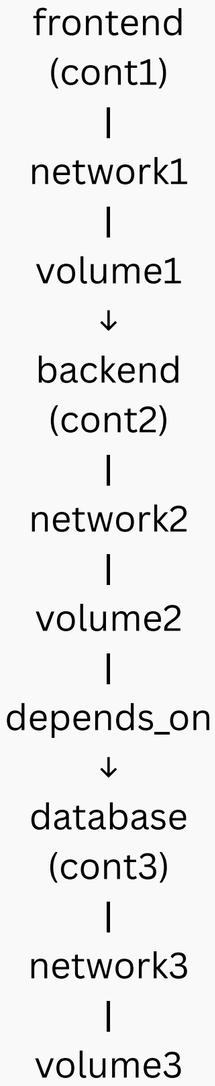
CPU Limit: 1.0

Memory Reservation: 100 MB

Memory Limit: 200 MB

Service Dependency

Backend depends on Database



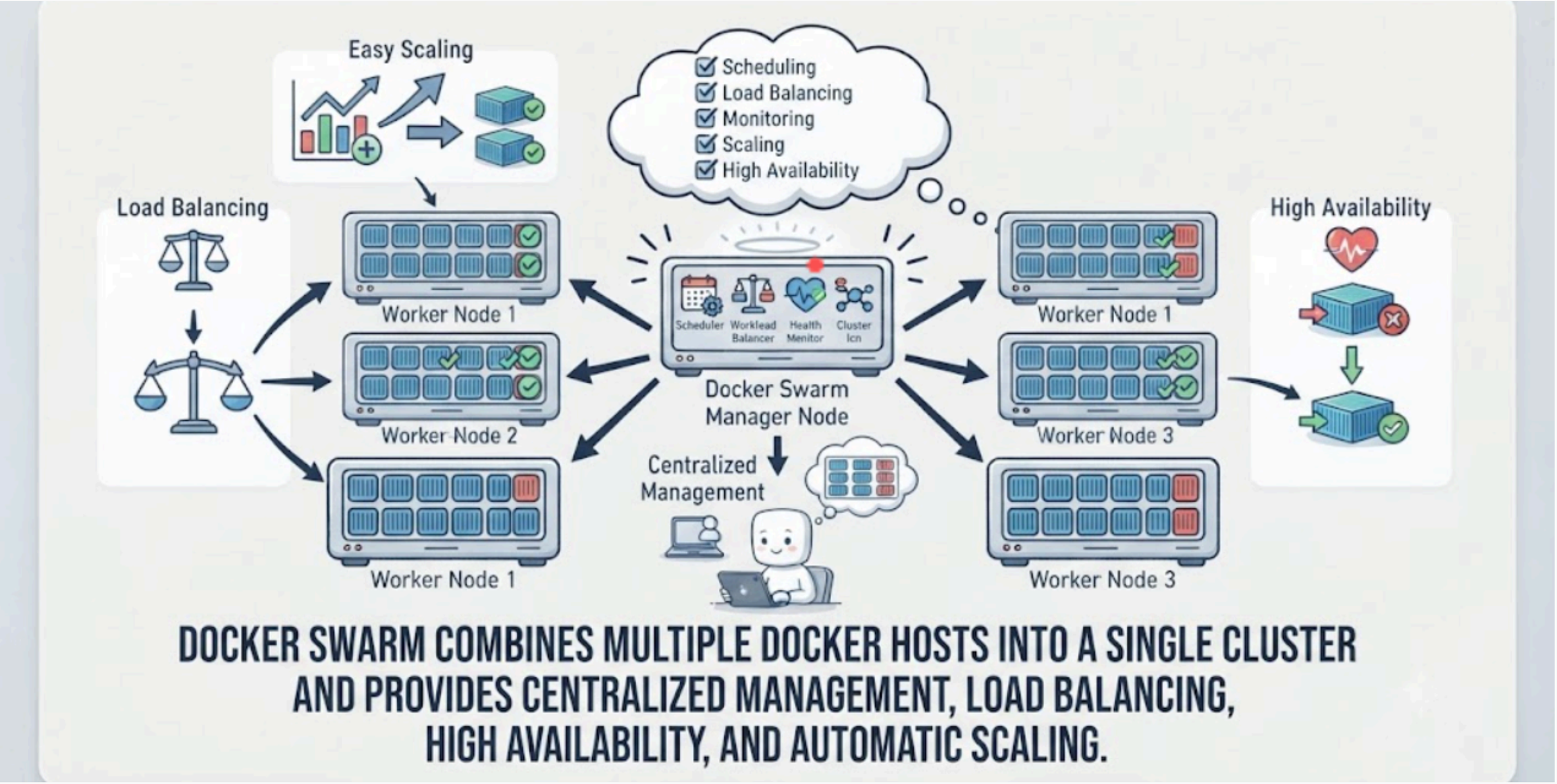
PROBLEMS WITH MULTIPLE CONTAINERS



WITHOUT DOCKER SWARM, MANAGING CONTAINERS MULTIPLE SERVERS IS DIFFIULT, TIME-CONSUMING.

SCALING BECOMES AN OPERATIONAL NIGHTMARE WITHOUT ORCHESTRATION.

Docker Swarm



EXPLANATION OF DOCKER SWARM

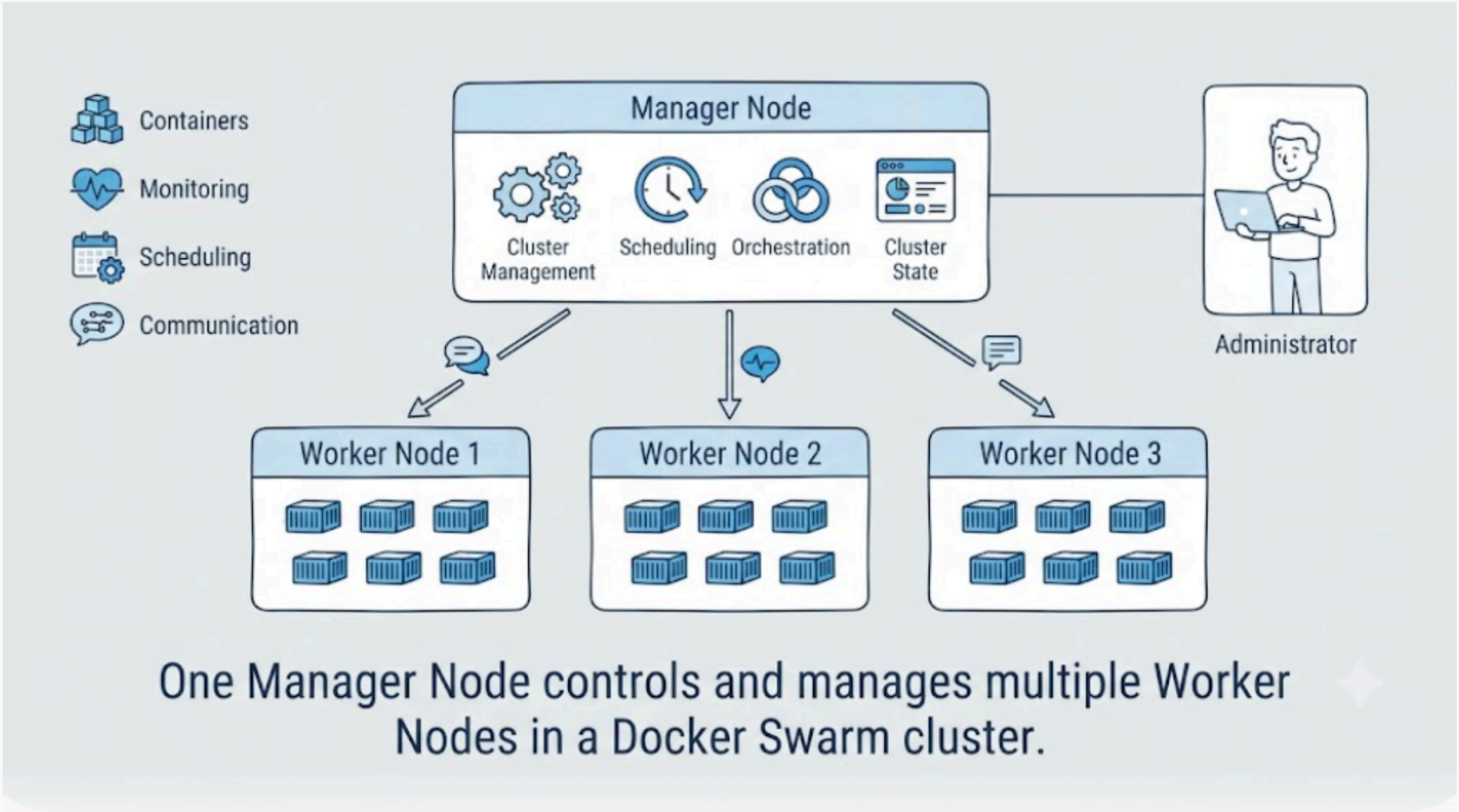
Docker Swarm simplifies the deployment, scaling, and management of containers across multiple Docker hosts.

KEY POINTS

- Combines multiple Docker hosts into a single cluster.
- Uses Manager and Worker nodes for cluster management.
- Enables centralized deployment and management of containers.
- Automatically distributes workloads across available nodes.
- Provides built-in load balancing for applications.
- Ensures high availability through container rescheduling.

Docker Swarm Architecture

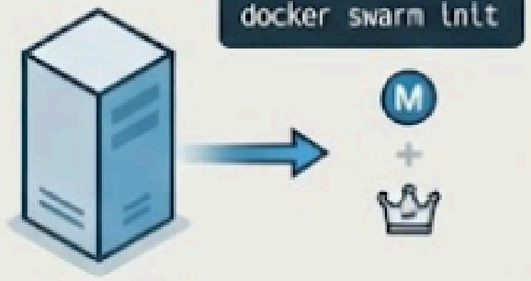
- Manager Node – Controls the swarm cluster, schedules services, maintains cluster state, and performs orchestration tasks.
- Worker Nodes – Execute containers assigned by the manager, process application workloads, and report status back to the manager.



Docker Swarm Setup

1 INITIALIZE SWARM

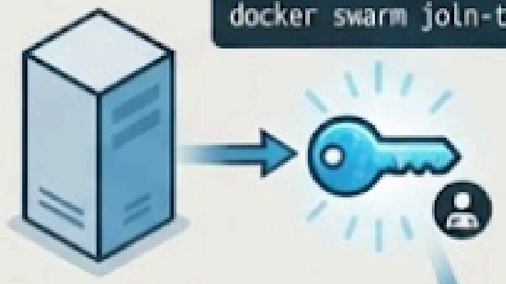
```
docker swarm lnit
```



Manager Node

2 GENERATE JOIN TOKEN

```
docker swarm joln-token worker
```



Manager Node

3 JORIFY CLUSTER

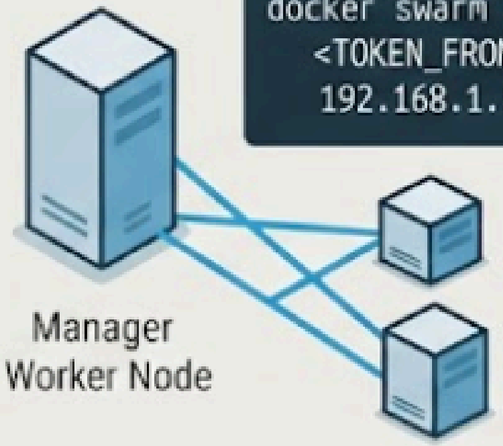
```
docker node ls
```

	Node	Status	Active
1	Manager	Ready	Active
3	Workers	Ready	Active
3	Workers	Ready	Active

Manager Node

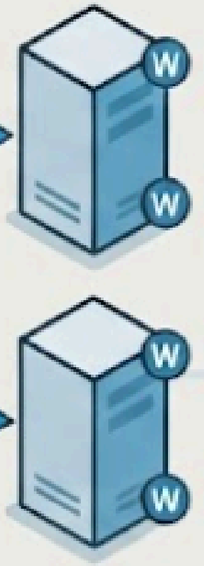
4 VERIFY CLUSTER

```
docker swarm join --token
  <TOKEN_FROM_MANAGER> \
  192.168.1.100:2377
```



Manager
Worker Node

Worker Node



Worker Nodes

Docker Swarm Setup

Step 1: Initialize Swarm Manager

Run the following command on the Manager Node:

Shell:docker swarm init

✅ **Creates a new Docker Swarm cluster and promotes the node as the Manager.**

Step 2: Generate Worker Join Token

Run on the Manager Node:

Shell:docker swarm join-token worker

✅ **Generates a token that allows worker nodes to join the swarm.**

Step 3: Join Worker Nodes

Run on each Worker Node:

Shell: docker swarm join --token <TOKEN> <MANAGER-IP>:2377

Parameters:

TOKEN → Worker join token generated by the manager

MANAGER-IP → IP address of the Manager Node

2377 → Docker Swarm management port

✅ **Adds the worker node to the swarm cluster.**

Step 4: Verify Cluster

Run on the Manager Node:

Shell:docker node ls

✅ **Displays all nodes participating in the swarm.**

Common Docker Swarm Commands

Get Manager Join Token

Shell:docker swarm join-token manager

View Swarm Information

Shell:docker info

Create a Service

Shell:docker service create --name nginx -p 80:80 nginx

List Services

Shell:docker service ls

Scale a Service

Shell:docker service scale nginx=5

Remove a Worker from Swarm

Shell:docker swarm leave

Force a Manager to Leave Swarm

Shell:docker swarm leave --force

Inspect a Node

Shell:docker node inspect <node-name>